

09 — Unrealizable Specifications

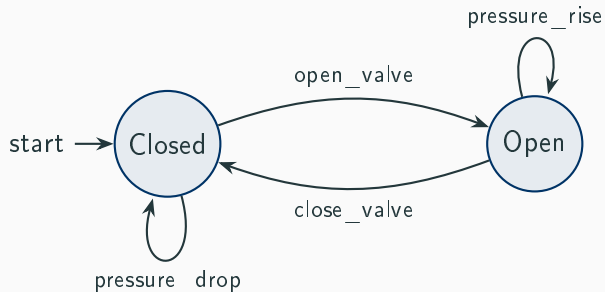
Diagnosis and Counterexamples

Mariano Cerrutti

Mununu Tutorial

The Valve Controller

A simple valve with two states and four events:



- **Controllable:** `open_valve`, `close_valve` (controller chooses).
- **Uncontrollable:** `pressure_rise`, `pressure_drop` (environment).

The Automaton in CTXDSL

```
1      automaton Valve {
2          controllable {
3              label open_valve;
4              label close_valve;
5          }
6
7          states {
8              state Closed initial;
9              state Open;
10         }
11
12         transitions {
13             transition Closed -> Open on label open_valve;
14             transition Open -> Closed on label close_valve;
15             // Environment events
16             transition Closed -> Closed on label pressure_drop;
17             transition Open -> Open on label pressure_rise;
18         }
19     }
```

An Impossible Specification

```
1  mu_formulas {
2      // UNREALIZABLE: demands being in both Open and Closed simultaneously.
3      // No state satisfies (Open && Closed) since they are distinct states.
4      formula impossible_spec {
5          over Valve;
6          body = Open && Closed;
7      }
8
9      // REALIZABLE: the valve can reach Open state from any state.
10     formula open_reachable {
11         over Valve;
12         body = mu X. (Open || <> X);
13     }
```

Open && Closed — no state is simultaneously Open *and* Closed.

Unsatisfiable (0/2 states).

$\mu X. (Open \vee \hat{\vee} X)$ — the valve can reach Open. Satisfiable (2/2 states)

Controller with Diagnostics

```
1  controllers {
2      // This controller will be UNREALIZABLE.
3      // Diagnostics reveal why: no initial state satisfies the spec.
4      controller impossible_valve {
5          source Valve;
6          satisfying impossible_spec;
7          diagnostics {
8              counterexample = true;
9              deadlock_traces = true;
10         }
11     }
```

Running synthesis with diagnostics enabled:

```
$ mununu context synth 09_unrealizable.ctxdsl \
  --controller impossible_valve
Controller impossible_valve: UNREALIZABLE
Violating initials: {Closed}
```

Formula Inversion

Inverting a formula computes the **environment's winning region** — the set of states from which the environment can violate the specification.

	Original	Inverted	Effect
	μ (mu)	ν (nu)	Reachability $\leftrightarrow$ Safety
open_reachable	ν (nu)	μ (mu)	Safety $\leftrightarrow$ Reachability
	[] (box)	$\langle \rangle$ (diamond)	Universal $\leftrightarrow$ Existential
	$\langle \rangle$ (diamond)	[] (box)	Existential $\leftrightarrow$ Universal
	ctrl=controllable	ctrl=environment	Controller $\leftrightarrow$ Environment

The inverted formula's satisfying states are exactly those where the **environment wins**.

Reference: Zielonka, W. (1998). "Infinite Games on Finitely Coloured Graphs with Applications to Automata on

Infinite Trees". *Theoretical Computer Science*

Lasso Traces

A **lasso trace** is a counterexample for liveness properties. It has the form:

$$\text{prefix} \rightarrow (\text{cycle})^\omega$$

Example (traffic light that never reaches Green):

$$\text{Red} \rightarrow (\text{PedWaiting})^\omega$$

- The **prefix** is a finite path from an initial state to the cycle entry.
- The **cycle** repeats forever, demonstrating that the goal is never reached.
- A counterexample for a liveness property $(G F p)$ is an infinite path that **avoids p from some point onward**.

Mununu reports lasso traces when `deadlock_traces` is enabled in diagnostics.

Counterstrategy Graph

When synthesis is unrealizable, Mununu can generate a **counterstrategy graph** — a subgraph showing how the environment wins.

CLI:

```
$ mununu context graph file.ctxdsl \  
  --counterstrategy \  
  --formula F --automaton A \  
  --output cs.html
```

UI: Click the “**Counterstrategy**” button in the Verification tab.

- The counterstrategy graph shows the **environment’s winning region** — states where the environment can force a violation.
- Edges represent the environment’s optimal moves.
- Use it to understand *which environment behaviors* make the spec unrealizable

Takeaway

Concept	Meaning
Unrealizable	No controller can enforce the specification — the property is too strong or contradictory.
<code>diagnostics</code>	Enables detailed output explaining failure.
<code>counterexample</code>	Shows a concrete witness of why synthesis fails.
<code>deadlock_traces</code>	Shows traces leading to states where the controller gets stuck.

Best practices:

- Always include a **realizable** formula for contrast (e.g., safety invariant).
- Enable diagnostics early — they are your debugger for specifications.
- An unrealizable result is not a tool failure; it means the spec needs revision.